COURSE CODE: BIIT 2305
COURSE TITLE: WEB APPLICATION DEVELOPMENT

PROJECT TITLE:

PLAYAQ: DIGITAL HOME MAINTENANCE & REPAIR PLATFORM

GROUP: 03

PREPARED FOR:
DR. NAJHAN BIN IBRAHIM

DATE OF SUBMISSION:
9 JUNE 2026

GITHUB LINK: https://github.com/balqeeshana/playaq-project-webapp

PREPARED BY:

NO.	NAME	MATRICS NO.
1	Anis Daniyah binti Mohd Faizal	2419018
2	Anis Asna bt Amin	2415028
3	Hana Imani Binti Jalaludin	2413760
4	Haifa Adani Binti Hanafiah	2415106
5	Balqees Hana Binti Fairuzam	2419962

TABLE OF CONTENTS

1.0 INTRODUCTION

In today's modern environment, maintaining essential household systems such as plumbing, electrical wiring, and air conditioning has become increasingly important. However, consumers frequently struggle to find reliable, certified, and trustworthy repair professionals. This operational gap often leads directly to negative customer experiences, financial loss, wasted time, and severe safety risks when unskilled or uncertified workers handle critical infrastructure.

To address these pain points, this project introduces **PLAYAQ,** a trusted, centralized digital platform engineered to safely connect users with thoroughly vetted and certified service professionals. By transitioning outdated, unverified manual inquiry processes into a structured digital ecosystem, PLAYAQ provides users with peace of mind and guarantees customer satisfaction across the entire transaction lifecycle.

2.0 PROJECT OBJECTIVES

To successfully deliver the PLAYAQ platform, the project focuses on achieving the following core objectives:

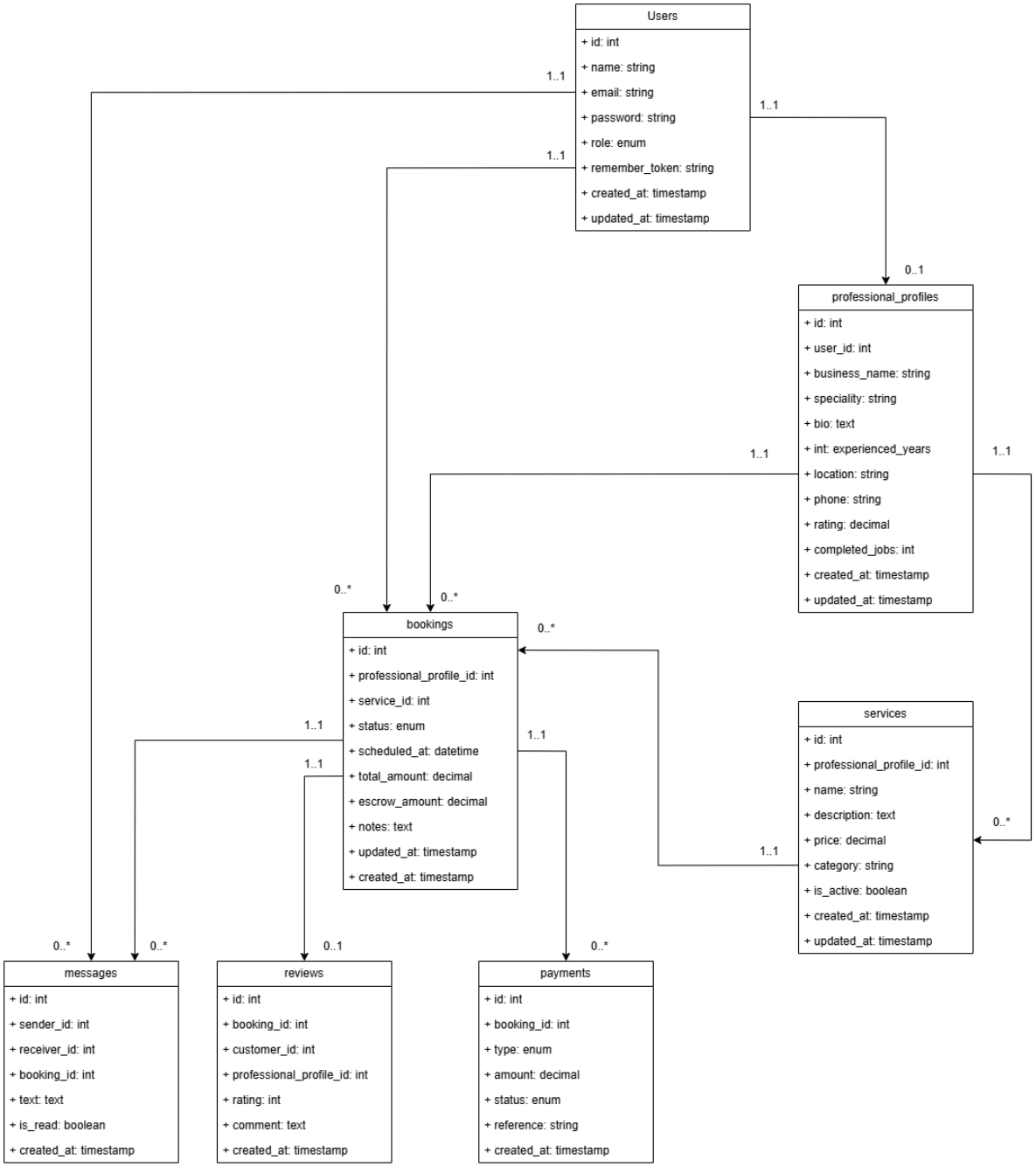
- **Build a Searchable Digital Repository**: Utilize Laravel's Eloquent ORM to construct a searchable database where users can seamlessly filter, evaluate, and view profiles of verified independent professionals across multiple service segments.
- **Provide a Transparent Booking System**: Implement a functional booking lifecycle featuring upfront pricing, transparent project quotations, and secure service records.
- **Establish Accountability and Quality Control**: Integrate a post-job review mechanism incorporating star ratings, user feedback, and warranty tracking to enhance provider accountability and service quality.
- **Create a Professional Portal**: Develop a dedicated business dashboard for technicians to manage their public profiles, showcase specialized badges, track earnings, and upload active vocational certifications.
- **Integrate an Interactive Assistant**: Build an interactive diagnostic assistant interface designed to guide users in identifying their specific repair issues and estimating costs prior to booking a professional.

3.0 FEATURES AND FUNCTIONALITIES

The PLAYAQ application provides distinct, secure workspaces for both service seekers and verified service providers. The system delivers the following functional elements:

- **Multi-Role User Management & Authentication**: Features distinct workflows for Customers and Service Professionals, securely sorting incoming traffic into specialized portals.
- **Geographical and Category Search**: An active discovery matrix allowing customers to filter out verified professionals by cross-referencing industry specialties (e.g., plumbing, painting, appliance repair) with fifteen distinct Klang Valley municipalities.
- **Dynamic Booking Engine with Pricing Thresholds**: Provides full booking lifecycles utilizing rule-based backup pricing engines to generate interactive financial estimates based on individual craft types.
- **Automated Escrow-Style Deposit Calculations**: Programmatically computes a locked 30% operational commitment fee down payment upon confirmation to shield micro-entrepreneurs from sudden customer cancellations.
- **Dynamic Earnings Analytics & Profiles**: A dedicated interface for professionals to track lifetime total payouts, current monthly ratios, pending assignments, and handle full CRUD manipulation of their service offerings.
- **Automated Trust & Quality Feedback System**: Integrated post-completion verification pipelines where customer feedback entries dynamically update professional aggregate stars and auto-increment total verified task counts.

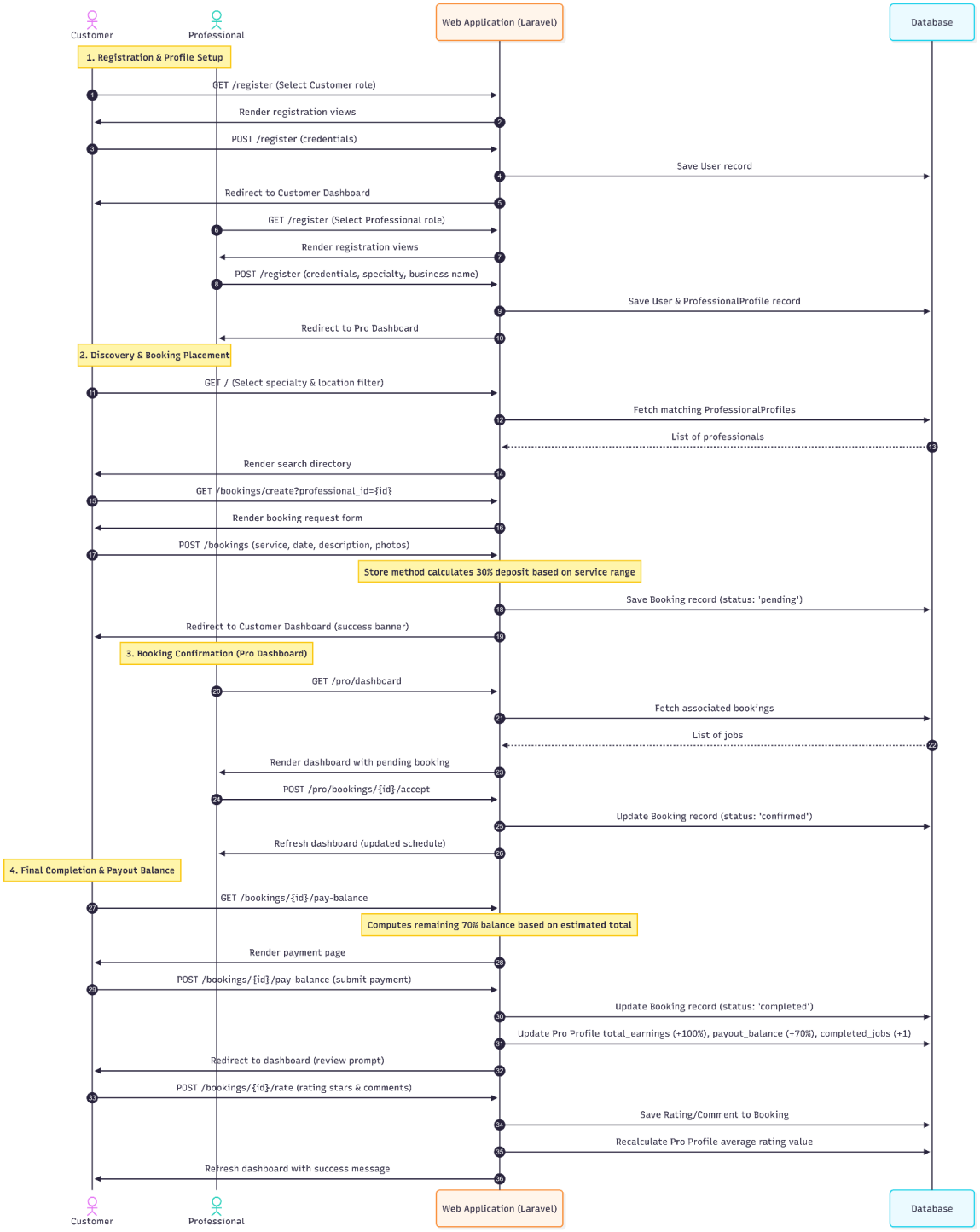
4.0 SYSTEM DESIGN



4.1 Core Data Modeling Definitions:

1. **users**: Root entity managing login attributes and security flags. Separates roles into customer and professional.
2. **professional_profiles** *(1:1 with users)*: Stores occupational metrics, trade backgrounds, locations, and real-time aggregate star evaluation records.
3. **services** *(1:N with professional_profiles)*: Holds customized target offerings and financial minimum/maximum limits controlled by the vendor.
4. **bookings** *(1:N with users as Customer, 1:N with professional_profiles)*: Core transactional state table housing schedule parameters, financial calculations, deposit metrics, and rating returns.
5. **messages** *(1:N with users via Sender/Receiver)*: Handles foundational data fields for text communications.

5.0 SYSTEM INTERACTION



6.0 TECHNICAL IMPLEMENTATION

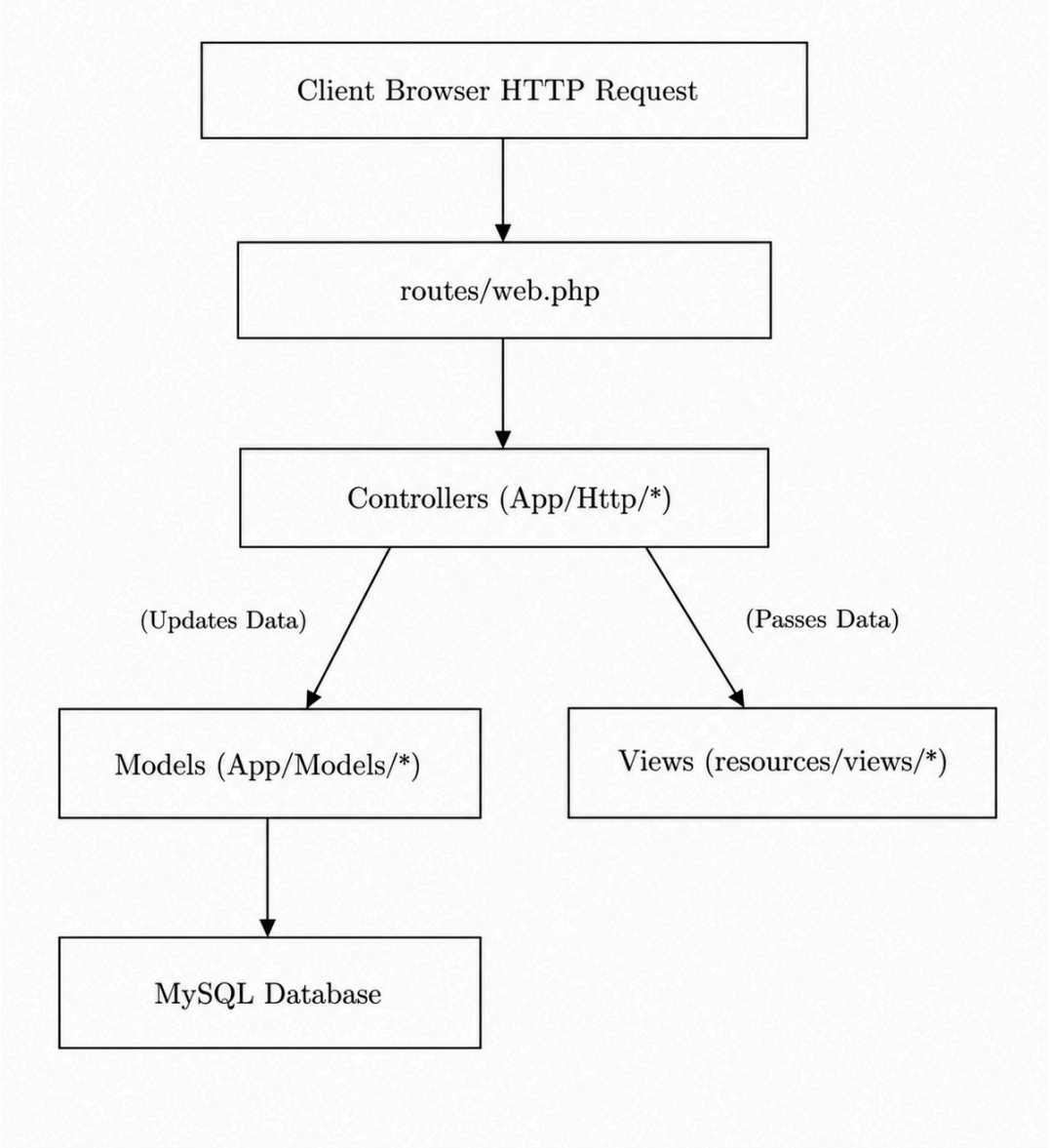
The PLAYAQ platform is developed using the following software stack and configuration settings:

- **Runtime Environment:** PHP 8.2+
- **Web Framework:** Laravel 11.x MVC Web Framework
- **Database Engine:** SQLite (configured in development for speed and simplicity) or MySQL
- **Front-End Styling:** Tailwind CSS (loaded via Play CDN)
- **UI Dynamics:** Alpine.js (Version 3.x) for responsive state management (menus, dropdowns)
- **Icon Library:** Lucide Icons (loaded via CDN)
- **Package Management:** Composer (PHP packages) and npm (Vite bundler configuration)
- **Vite Integration:** Dynamic loading of custom styles using Vite config and @vite directives.

7.0 MVC ARCHITECTURE

PLAYAQ follows Laravel's **Model-View-Controller (MVC)** architecture, which helps keep the system organized by separating different responsibilities:

- **Models** manage data and database operations.
- **Views** handle the user interface and what users see.
- **Controllers** process requests and connect the Models and Views.



7.1 Model Component (app/Models/*)

The Model layer is responsible for managing the application's data, business entities, and database rules. Using Laravel's Eloquent ORM, database records are represented as objects, making data management easier and reducing the need for raw SQL queries while maintaining data integrity.

1. **User.php**
    - Manages user account information and authentication.
    - Defines relationships with bookings, messages, and professional profiles.
    - Handles basic account roles and access permissions.

2. **ProfessionalProfile.php**
    - Stores information about service providers and their businesses.
    - Connects to user accounts, services offered, and customer bookings.
    - Maintains business details such as ratings, statistics, and payout balances.

3. **Service.php**
    - Represents the services offered by professionals.
    - Links each service to its corresponding professional profile.
    - Stores service details, including minimum and maximum pricing ranges.

4. **Booking.php**
    - Manages the entire booking process between customers and professionals.
    - Tracks booking status updates throughout the transaction lifecycle.
    - Handles uploaded booking photos by converting stored JSON data into usable PHP arrays.

5. **Message.php**
    - Manages the internal messaging system.
    - Stores sender and receiver information using foreign key relationships.
    - Maintains chat histories and message records between users.

7.2 View Component (resources/views/*)

The View layer handles the application's presentation tier, dictating how information is structured and displayed to end-users. Powered by the Blade Templating Engine, views safely consume variables passed from the controller without ever executing direct queries against the database:

- **layouts/app.blade.php**: The global master template skeleton defining structural script headers, responsive mobile navigation layouts, alert banner slots, and common footers.
- **home.blade.php**: The public marketplace discovery interface containing regional/specialty search filters and dynamic professional overview grids.
- **booking/**: Dedicated sub-views managing booking creation forms, localized service selection sheets, and simulated invoice settlement pages.
- **customer/**: Specialized workspace panels housing active service booking histories, progress-tracking tables, and profile modification menus.
- **professional/**: A unified business dashboard displaying provider revenue analytics, work schedule lists, custom service CRUD options, and certification upload inputs.
- **chat/**: Secure direct messaging thread windows displaying clean, chronological text dialogue interfaces between system users.

7.3 Controller Component (app/Http/Controllers/*)

The Controller layer represents the intermediary operational brain of the application. It handles incoming client HTTP requests, enforces input data validation rules, triggers state updates across our models, and passes the computed outputs straight to the appropriate presentation view:

- **AuthController.php**: Manages customer and professional account registration pipelines, processes secure login sessions, and performs secure token invalidation during logouts.
- **HomeController.php**: Intercepts active search parameters to query professional marketplace listings based on distinct specialties and location tags.
- **BookingController.php**: Orchestrates the multi-stage booking lifecycle, executing automated 30% down-payment calculations, balance payment resolutions, and post-completion star-rating updates.

- **ProfessionalController.php**: Aggregates dashboard career metrics, handles full CRUD operations for individual trade services, updates business details, and executes secure file uploads for technical vocational certifications.
- **ProfileController.php**: Manages regular customer workspace data configurations and individual profile modifications.
- **ChatController.php**: Sanitizes, records, and fetches peer-to-peer text exchanges to drive the live system messaging stream.

8.0 ROUTES & CONTROLLERS

In PLAYAQ, the routing layer acts as the main entry point to the application. It directs incoming HTTP requests to the correct backend functions while helping maintain security. The controller layer processes user requests, validates input data, communicates with database models, and manages different workflows for customers and service providers.

8.1 Routing Architecture

All application routes are defined in **routes/web.php** and organized into three main categories to ensure proper access control and security.

1. **Public Routes**

- Accessible to all users, including visitors who are not logged in.
- Allow users to browse service providers and view public information.

2. **Guest Middleware Routes**

- Only accessible to users who are not logged in.
- If a logged-in user tries to access these routes, they are automatically redirected to their dashboard.

3. **Protected Authentication Routes**

- Require users to have a valid login session.
- Protect important features such as:
    - User dashboards
    - Messaging system
    - Service management
    - Booking and order processing

8.2 Controller Breakdown

a. **AuthController:** The AuthController handles user authentication and account management.

Main Functions:

- **showLoginForm() and showRegisterForm()**
  - Check whether a user is already logged in.
  - Redirect active users away from login and registration pages.
- **login()**
  - Verifies user credentials using Auth::attempt().
  - Regenerates the session ID to improve security and prevent session-fixation attacks.
- **register()**
  - Creates a new user account.
  - If the user registers as a professional, an additional professional profile is created.
  - Automatically logs the user into the system after registration.

b. **BookingController:** The BookingController manages the booking and payment process within the marketplace.

Main Functions:

- **store()**
  - Processes booking requests.
  - Calculates a required 30% deposit based on the selected service's pricing range.
  - Saves the booking with a pending status.
- **showPayBalanceForm()**
  - Calculates the remaining 70% balance that needs to be paid.
- **payBalance()**
  - Processes the final payment.
  - Updates the booking status to completion.
  - Updates the professional's earnings.

- **rateBooking()**
    - Records customer ratings and reviews.
    - Updates the professional's overall rating.

c. **ProfessionalController:** The ProfessionalController helps service providers manage their business profiles and services.

Main Functions:

- **index()**
    - Displays business information such as earnings and active bookings.
    - Generates dashboard analytics for professionals.
- **updateProfile()**
    - Allows professionals to update their business profile and biography.
- **addService() and deleteService()**
    - Manage service listings offered by professionals.
- **withdraw()**
    - Simulates withdrawing earnings by resetting the available payout balance.
- **uploadCertificate()**
    - Allows professionals to upload certifications.
    - Enforces a maximum file size limit of 10MB before storing the file securely.

8.3 Route-to-Controller Master Mapping Matrix

HTTP Method	Route / URL Path	Target Controller Action	Bound Middleware	Operational Purpose / Business Logic
GET	/	HomeController@index	web	Marketplace discovery page; processes query-string filters.

GET	/supplier-partnerships	Closure Template	web	Renders the static supplier and merchant partnership page.
GET	/login	AuthController @showLoginForm	guest	Displays the user login screen; redirects active sessions.
POST	/login	AuthController @login	guest	Validates credentials, executes login, and switches session IDs.
GET	/register	AuthController @showRegisterForm	guest	Renders user registration page containing role selection blocks.
POST	/register	AuthController @register	guest	Persists User and Profile records, then logs the user in.
POST	/logout	AuthController @logout	auth	Flushes session variables, invalidates tokens, and signs out users.
GET	/profile	ProfileController@show	auth	Displays customer account overview and active personal configurations.
POST	/profile/update	ProfileController@update	auth	Updates and patches basic customer personal contact details.
GET	/instant-booking	BookingContro	auth	Loads the unified

		ller@showInstantBookingForm		instant step-by-step booking selection wizard.
GET	/chat/{receiver_id}	ChatController@show	auth	Fetches and displays chronological private direct messaging threads.
POST	/chat/send	ChatController@send	auth	Validates and saves incoming text data to the messages log table.
GET	/dashboard	BookingController@customerDashboard	auth	Loads the customer panel and gathers current booking histories.
GET	/bookings/create	BookingController@create	auth	Displays the service reservation request scheduling page.
POST	/bookings	BookingController@store	auth	Calculates the 30% platform commitment deposit and saves row.
GET	/bookings/{id}/pay	BookingController@showPayBalanceForm	auth	Processes transaction loops to extract remaining 70% balances.
POST	/bookings/{id}/pay	BookingController@payBalance	auth	Processes final settlement, closing jobs and allocating earnings.
POST	/bookings/{id}/rate	BookingContro	auth	Saves job feedback

		ller@rateBooking		scores and updates aggregate rating loops.
GET	/pro/dashboard	ProfessionalController@index	auth	Gathers analytics, statistics, and job orders for professionals.
POST	/pro/profile/update	ProfessionalController@updateProfile	auth	Patches professional storefront records, descriptions, and locations.
POST	/pro/services/add	ProfessionalController@addService	auth	Appends custom items into the professional's service inventory.
DELETE	/pro/services/{id}	ProfessionalController@deleteService	auth	Removes a selected service entry package from the database layer.
POST	/pro/bookings/{id}/accept	BookingController@acceptBooking	auth	Confirms the booking status from pending to confirmed.
POST	/pro/withdraw	ProfessionalController@withdraw	auth	Clears virtual wallet balances and runs simulated bank cashouts.
POST	/pro/certificate/upload	ProfessionalController@uploadCertificate	auth	Validates and saves vocational trade qualification media files.

9.0 MODELS & VIEWS

In PLAYAQ, the decoupled relationship between the data persistent layer (Models) and the presentation tier (Views) serves as the core foundation of the platform. By leveraging Laravel's Eloquent ORM, database records and complex multi-entity relationships are abstracted into clean object-oriented code. Simultaneously, the user interface utilizes the Blade Templating Engine to render dynamic data while remaining completely isolated from raw queries and background arithmetic logic.

9.1 Database Models & Eloquent Relationships (app/Models/*)

Database transactions and constraints are managed via native Eloquent model mappings rather than writing volatile raw SQL statements. Relationship boundaries across the system are configured as follows:

- **User.php**: Serves as our central identity and authentication table. It maintains a One-to-One structural bridge via professionalProfile() (hasOne(ProfessionalProfile::class)) to separate standard household customers from trade service providers. It records client-side order patterns via bookings() (hasMany(Booking::class, 'customer_id')) and tracks direct communication channels using sentMessages() and receivedMessages().
- **ProfessionalProfile.php**: Represents the digital storefront and financial balance book for verified handymen. It links backwards to its identity record using user() (belongsTo(User::class)), maps custom service configurations using services() (hasMany(Service::class)), and aggregates customer assignments via bookings() (hasMany(Booking::class, 'professional_profile_id')).
- **Service.php**: Defines the modular trade offerings customized by an independent professional. Each listing binds inversely to its parent vendor storefront using professionalProfile() (belongsTo(ProfessionalProfile::class)) and enforces baseline minimum and maximum price constraints.
- **Booking.php**: The central transactional engine that handles tracking states. It references the hiring client via customer() (belongsTo(User::class, 'customer_id')) and the assigned contractor via professionalProfile() (belongsTo(ProfessionalProfile::class)). To support multiple job-site media uploads, it includes an array-casting layer (protected $casts = ['photo_paths' => 'array']) to automatically handle JSON string conversion.

- **Message.php**: Manages the underlying schema for peer-to-peer real-time text logs. It tracks conversation histories by mapping sender() and receiver() attributes back to independent foreign keys pointing to the core User model.

9.2 Front-End Views & Blade Engine (resources/views/*)

The presentation layer is kept completely isolated from operational business logic, relying on variables passed down from the controllers to render interfaces via clean Blade layout structures:

- **layouts/app.blade.php**: The foundational global wrapper that bundles global Tailwind CSS styling libraries, Alpine.js responsive interaction scripts, dynamic navigational menus, and the footer grid.
- **home.blade.php**: The public marketplace catalog displaying active trade provider cards alongside live query filters based on Klang Valley regional locations and handyman specialties.
- **booking/create.blade.php**: The client-side appointment scheduler and service request configuration interface.
- **booking/pay_balance.blade.php**: The simulated checkout settlement invoice displaying item breakdowns, deposit deductions, and final payment collection inputs.
- **customer/dashboard.blade.php**: The home client portal providing chronological lists of requested home services, real-time job status tracking badges, and feedback submission forms.
- **customer/profile.blade.php**: Secure input form layout enabling users to modify their personal account contact parameters.
- **professional/dashboard.blade.php**: A unified business dashboard summarizing lifetime earnings, active work calendar schedules, trade service package CRUD tables, and vocational certificate upload forms.
- **chat/show.blade.php**: The direct messaging layout rendering private, chronological text conversation panels between customers and providers.

9.3 Data Model to View Mapping Matrix

Model Entity	Database Table	Main Eloquent Relationships	Target Blade Views	Operational Purpose & UI Representation
User	users	hasOne(ProfessionalProfile) hasMany(Booking) hasMany(Message)	auth/login.blade.php auth/register.blade.php customer/profile.blade.php	Manages credential security, active session tracking states, and personal customer profiles.
ProfessionalProfile	professional_profiles	belongsTo(User) hasMany(Service) hasMany(Booking)	home.blade.php professional/dashboard.blade.php	Stores public business names, biographies, trade specialties, aggregate ratings, and digital wallet balances.
Service	services	belongsTo(ProfessionalProfile)	professional/dashboard.blade.php booking/create.blade.php	Represents custom service offerings, task descriptions, and minimum/maximum budget thresholds.
Booking	bookings	belongsTo(User, 'customer_id') belongsTo(ProfessionalProfile)	customer/dashboard.blade.php booking/create.blade.php booking/pay_bala	Coordinates scheduling statuses, deposit requirements, milestone calculations, and user review

			nce.blade.php	comments.
Message	messages	belongsTo(User, 'sender_id') belongsTo(User, 'receiver_id')	chat/show.blade.php	Stores secure text dialogue records and timestamps driving the peer-to-peer communication thread.

10.0 USER AUTHENTICATION

10.1 Authentication Mechanism

PLAYAQ implements a custom, session-based authentication system built on Laravel's core auth services. The system does not rely on heavy starter kits like Jetstream or Livewire but instead, it utilizes custom controllers and standard Blade views to maintain a lightweight codebase:

- **Session Security:** Upon successful login via Auth::attempt, the user session is regenerated using $request->session()->regenerate() to protect against Session Fixation attacks.
- **Password Encryption:** All passwords are encrypted during registration using Hash::make() which uses the secure Bcrypt algorithm.
- **Role Verification & Navigation:** The system evaluates user roles (customer or professional) during login and registration and redirects them to their respective dashboards.
- **Route Protection:** Standard Laravel middlewares (auth and guest) protect routes. Guest-only routes prevent logged-in users from re-accessing login forms.
- **Secure Sign Out:** Logouts use $request->session()->invalidate() and $request->session()->regenerateToken() to clear all session variables and prevent CSRF replay attacks.

10.2 Database User Schema & Security

The authentication system relies on the following database fields in the users table:

- email (VARCHAR, UNIQUE): Serves as the primary identifier for authentication.
- password (VARCHAR): Stores the 60-character Bcrypt hash.
- role (VARCHAR): Determines access privileges (either customer or professional).
- remember_token (VARCHAR, NULLABLE): Enables persistent login sessions.

11.0 DESIGN & UI

This section highlights the user interface design choices, layout structure, and navigation flows applied to ensure a seamless, professional, and accessible user experience across PLAYAQ.

11.1 Design System & Aesthetic Rationale

PLAYAQ implements a modern, premium design system engineered using the **Tailwind CSS** framework to establish an authoritative visual hierarchy:

- **Color Palette:** The UI uses a carefully chosen color palette consisting of brand-50 (#f5f3ff, Lavender Tint), brand-600 (#4f46e5, Indigo), brand-700 (#4338ca, Deep Indigo), slate-50 (#f8fafc, Off-White/Ghost White), and slate-900 (#0f172a, Deep Slate/Charcoal). These colors form the overall visual identity of the interface.
- **Typography:** The Google Font **Outfit** is imported globally across the application. It offers a clean, geometric, and highly modern sans-serif aesthetic that ensures readability at any scale.
- **Layout & Grid Architecture:** The interface combines the utility of Bootstrap structures with a flexible Tailwind CSS grid framework. Core layouts leverage structured Bootstrap container configurations alongside Tailwind's centralized max-width utilities (max-w-7xl). This multi-framework approach enforces rigid, clean alignment, keeping content bounded and visually uncluttered across all interface views.

11.2 UI Theme & User Experience (UX)

The application prioritizes user intuition and cognitive ease through deliberate interface choices:

- **Aesthetic Styling & Glassmorphism:** To elevate the premium aesthetic without adding visual noise, the global header features an 80% translucent white background (bg-white/80)

combined with a backdrop blur effect (backdrop-blur-md). This allows it to float cleanly over content and remain distinct during scrolling actions.

- **Cognitive Load Reduction:** By using a clean, simple background style (slate-50) and generous whitespace, layouts remain intuitive. This intentional minimalism prevents user confusion and focuses attention purely on functional workflows.
- **Responsive Design:** Built entirely with a responsive design philosophy, the Blade layouts adapt dynamically across all user devices. Grid columns, navigation panels, and data tables seamlessly reflow from compact portable viewports up to expansive desktop monitors without losing layout integrity.

11.3 Navigation Structure & Links

The platform utilizes a structured navigation ecosystem to keep the user perfectly oriented at all times:

- **Global Navigation Bar:** The primary header navbar handles core routing paths. It dynamically adapts based on the user's authentication state:
    - Guest State: Displays public marketing pages, platform information, and clear “Login” / “Register” calls to action.
    - Authenticated State: Transforms to show context-aware links, user profile shortcuts, and role-specific dashboard entries.
- **Sidebar & Dashboards:** Secondary navigation panels utilize dedicated sidebars for dense administrative environments (such as student, admin, or provider dashboards). Dropdown menus and navigation drawers utilize **Alpine.js** directives (x-show, x-transition) for fluid, animated click-away interactions.
- **Breadcrumbs & Hyperlinks:** Every internal view is supported by consistent hyperlink naming conventions and logical navigation paths. Breadcrumbs ensure users can trace their steps backward effortlessly, eliminating dead ends.
- **Feedback & Session Alerts:** To guide user behavior interactively, the layout employs bright, high-visibility feedback banners using Blade session alerts (vibrant green for success messages, deep red for validation errors), providing immediate validation for every user action.

12.0 MEDIA ELEMENTS

PLAYAQ integrates a variety of media elements to enrich the user experience:

- **Brand Iconography:** The platform uses a customized branding logo asset (images/logo.png) displayed in headers, footers, and authentication screens.
- **Scalable Vector Graphics:** Standard UI actions (e.g. booking, chat, calendars, logs) display scalable vector graphics icons powered by the Lucide Icons library.
- **Dynamic Booking Attachments:** Customers can upload multiple images when submitting booking requests (stored inside bookings.photo_paths JSON field).
- **Professional Certificate Uploads:** Handymen can upload vocational certifications. The files (PDF, PNG, JPG, JPEG) are validated up to 10MB on the server and verified status indicators are displayed.

13.0 REFERENCES

1. *Alpine.js*. (n.d.). Alpinejs.dev. https://alpinejs.dev/
2. Angi - Home for everything home. (n.d.). *Angi*. https://www.angi.com/
3. Bootstrap. (2025). *Bootstrap*. Getbootstrap.com. https://getbootstrap.com/
4. tailwindcss. (2025). *Tailwind CSS - Rapidly build modern websites without ever leaving your HTML.* Tailwindcss.com. https://tailwindcss.com/
5. **Ibrahim, N. M.** (n.d.). *Blade template* [Lecture slides]. BIIT 2305 Web Application Development, Kulliyyah of ICT, International Islamic University Malaysia.
6. **Ibrahim, N. M.** (n.d.). *Chapter 11: Model* [Lecture slides]. BIIT 2305 Web Application Development, Kulliyyah of ICT, International Islamic University Malaysia.
7. **Ibrahim, N. M.** (n.d.). *Controller* [Lecture slides]. BIIT 2305 Web Application Development, Kulliyyah of ICT, International Islamic University Malaysia.
8. **Ibrahim, N. M.** (n.d.). *Laravel MVC model* [Lecture slides]. BIIT 2305 Web Application Development, Kulliyyah of ICT, International Islamic University Malaysia.
9. Laravel. (2025). *Laravel documentation*. https://laravel.com/docs
10. W3Schools. (2025). W3Schools online web tutorials. W3schools.com; W3Schools. https://www.w3schools.com/